

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

МОСКОВСКИЙ ИНСТИТУТ ЭЛЕКТРОНИКИ И МАТЕМАТИКИ
им. А.Н. ТИХОНОВА

Программная платформа для поддержки принятия решений

Руководство разработчика

Москва, 2026

Оглавление

1. Назначение документа	3
2. Основания для разработки и краткая характеристика системы	3
3. Архитектура программного средства	3
4. Технологический стек и причины выбора	4
5. Структура проекта	4
6. Описание серверной части	5
7. Описание клиентской части	6
8. Хранение данных и файлов	6
9. Безопасность и разграничение доступа	6
10. Развёртывание и сопровождение	7
11. Порядок доработки и расширения системы	7
12. Заключение	8

1. Назначение документа

Настоящее руководство разработчика содержит сведения, необходимые для понимания внутреннего устройства, доработки, сопровождения и развёртывания информационной системы управления конференцией. Документ адресован разработчикам, сопровождающим специалистам, техническим руководителям и членам комиссии, анализирующим архитектурные решения проекта.

2. Основания для разработки и краткая характеристика системы

Система разработана как веб-приложение для автоматизации процессов подготовки и проведения научных конференций. Функциональная область включает учёт пользователей, распределение ролей, создание конференций и секций, приём материалов, управление рецензированием, вынесение решений по заявкам и формирование программы.

Система реализована по модели SPA + REST API. Клиентская часть отвечает за пользовательский интерфейс и сценарии взаимодействия, а серверная часть отвечает за бизнес-логику, безопасность, работу с базой данных, файловым хранилищем и уведомлениями.

3. Архитектура программного средства

Архитектура проекта многослойная. Внешний пользователь работает через браузер с одностраничным приложением на React. HTTP-запросы проходят через Nginx, который маршрутизирует обращения либо на фронтенд, либо на серверный API. Backend реализован на FastAPI и взаимодействует с PostgreSQL для хранения структурированных данных и с MinIO для хранения файлов конференционных материалов.

В состав системы входят следующие логические подсистемы:

1. Пользовательский интерфейс, реализованный в каталоге frontend.
2. Прикладной сервер и бизнес-логика, реализованные в каталоге backend.
3. Реляционная база данных PostgreSQL.
4. Объектное файловое хранилище MinIO.
5. Прокси-сервер Nginx.
6. Контейнерная среда запуска Docker Compose.

4. Технологический стек и причины выбора

В серверной части используется Python 3 и фреймворк FastAPI. Python выбран как язык с высокой скоростью разработки и хорошей читаемостью, а FastAPI — как современный фреймворк для построения REST API с автоматической валидацией данных и удобной системой зависимостей.

Для работы с базой данных используется SQLAlchemy. ORM позволяет описывать предметную область через классы моделей и поддерживать ясное разделение между бизнес-логикой и доступом к данным.

В качестве основной СУБД выбрана PostgreSQL, поскольку система содержит большое количество взаимосвязанных сущностей: пользователей, конференций, ролей, секций, заявок, рецензий и приглашений. PostgreSQL обеспечивает надёжность, поддержку UUID и механизмы реляционной целостности.

Для хранения файлов используется MinIO. Это решение позволяет не загружать БД бинарными объектами и поддерживать независимое хранение статей, тезисов и версий материалов.

Клиентская часть построена на React и собирается через Vite. React выбран как библиотека для построения SPA с компонентной моделью и управлением состоянием, а Vite — как быстрый инструмент локальной разработки и сборки.

Для стилизации используется Tailwind CSS. Он обеспечивает быстрый конструируемый интерфейс, удобный для проектирования ролеориентированных экранов системы.

Авторизация реализована через JWT. Такой подход удобен для SPA-приложений и позволяет хранить состояние аутентификации на клиенте без серверных сессий.

5. Структура проекта

Структура репозитория организована по подсистемам:

1. Каталог backend содержит модели, схемы, сервисы, репозитории, API-маршруты и код авторизации.
2. Каталог frontend содержит компоненты интерфейса, представления, API-клиент и стили.
3. Каталог nginx содержит конфигурацию внешнего прокси-сервера.

4. Файл `docker-compose.yml` описывает совместный запуск инфраструктурных сервисов.

5. Файл `.env.example` содержит шаблон переменных окружения.

6. Описание серверной части

6.1. Точка входа и маршрутизация

Точка входа серверной части — файл `backend/main.py`. Здесь создаётся экземпляр приложения FastAPI, подключается CORS, регистрируются маршруты модулей `auth`, `conferences`, `submissions`, `reviews`, `files`, `notifications` и служебные маршруты контроля работоспособности.

6.2. Модуль авторизации

Модуль `auth` отвечает за регистрацию пользователей, вход в систему, получение текущего пользователя и регистрацию по приглашению. JWT формируется в `backend/auth/utlis.py`. После успешной аутентификации токен содержит идентификатор пользователя и срок действия. Извлечение пользователя из токена выполняется в `backend/auth/dependencies.py`.

Контроль доступа реализован через `dependency`-функции `require_superuser` и `require_conference_role`. Таким образом, критичные проверки выполняются на серверной стороне и не зависят от интерфейса клиента.

6.3. Слой моделей и схем

Модели SQLAlchemy описывают основные сущности предметной области: `User`, `Conference`, `ConferenceRole`, `Section`, `Submission`, `SubmissionAuthor`, `File`, `SubmissionFile`, `ReviewAssignment`, `Review` и `Invite`.

Схемы Pydantic описывают формат входящих данных и используются для валидации на границе API. Это упрощает контроль целостности данных и централизует проверку полей, таких как `email`, пароли, `UUID` и параметры операций.

6.4. Сервисный слой

Основная бизнес-логика вынесена в каталог `backend/services`. Такой подход снижает связность между HTTP-маршрутами и предметной логикой. Маршруты принимают и возвращают данные, а сервисы реализуют реальные сценарии: создание конференции, приглашение участников, обработку заявок, назначение рецензентов, загрузку версий файлов и отправку уведомлений.

6.5. Репозиторный слой

Репозитории инкапсулируют операции чтения и записи данных. Это упрощает сопровождение кода и позволяет расширять логику доступа к данным без изменения прикладных сервисов.

7. Описание клиентской части

Клиентская часть построена как одностраничное приложение. Главный файл `frontend/src/App.jsx` содержит центральную координацию состояния: проверку факта авторизации, загрузку профиля, списка конференций, определение роли в конференции и отображение нужного представления.

В приложении используются хуки React `useState`, `useEffect` и `useMemo`. `useState` хранит текущее состояние интерфейса, `useEffect` загружает данные по мере изменения контекста, а `useMemo` применён для вычисления производных данных и уменьшения лишних перерасчётов.

Клиентская часть разделена на представления по ролям: `AuthorView`, `ReviewerView`, `ChairmanView` и административные представления. Такое разделение отражает реальную ролевую модель предметной области.

Обращение к backend централизовано через слой `frontend/src/api`. Базовые функции `apiRequest` и `apiFetch` добавляют JWT-токен, обрабатывают JSON и `FormData`, интерпретируют ошибки сервера и предоставляют единый способ работы с API.

8. Хранение данных и файлов

Структурированные данные хранятся в PostgreSQL. Особенно важной сущностью является `SubmissionFile`, поскольку она реализует хранение типов файлов и версий материалов, что необходимо для прохождения заявкой этапов доработки.

Файлы размещаются в MinIO. При загрузке backend проверяет допустимый MIME-тип, формирует путь хранения, передаёт поток файла в объектное хранилище и сохраняет метаданные в таблице `files`.

Скачивание выполняется либо по временной ссылке, либо прямой отдачей потока. При этом проверка прав доступа выполняется до получения файла.

9. Безопасность и разграничение доступа

В проекте применён принцип, согласно которому интерфейс не является доверенной средой. Все критичные проверки выполняются на сервере. К ним относятся проверка токена, определение пользователя, проверка роли,

контроль принадлежности файла пользователю, проверка возможности назначения рецензента и прав на изменение статусов.

Пароли хранятся не в открытом виде, а в виде хеша. Для этого используются `passlib` и `bcrypt`. Это обеспечивает базовую стойкость к компрометации учётных записей при утечке базы данных.

JWT-токен используется для подтверждения личности пользователя, а таблица `conference_roles` — для определения его полномочий в конкретной конференции.

10. Развёртывание и сопровождение

Развёртывание выполняется через `Docker Compose`. В составе конфигурации поднимаются сервисы `PostgreSQL`, `MinIO`, `Redis`, `backend`, `frontend` и `Nginx`. Такой подход обеспечивает воспроизводимое окружение, упрощает локальную разработку и демонстрацию проекта.

`Backend` собирается на базе `Python 3.11`, `frontend` — на базе `Node.js 20 Alpine`. `Nginx` принимает внешний HTTP-трафик и разделяет маршрутизацию между пользовательским интерфейсом и API.

Для сопровождения рекомендуется вести изменения через сервисный и репозиторийный слои, а также документировать изменения схемы данных. В текущем варианте проекта используется автоматическое создание таблиц при старте и частичное изменение схемы на уровне приложения, что удобно для учебной разработки, но в промышленной эксплуатации обычно заменяется полноценными миграциями.

11. Порядок доработки и расширения системы

Рекомендуемый порядок внесения изменений следующий:

1. Сформулировать изменение предметного правила или пользовательского сценария.
2. При необходимости добавить или изменить `Rydantic`-схему.
3. Реализовать новую бизнес-логику в сервисном слое.
4. Дополнить репозитории и модели базы данных при необходимости.
5. Расширить API-маршрут.
6. Подключить новый сценарий на фронтенде через слой `api` и соответствующее представление.

Если дорабатывается ролевая логика, необходимо проверять согласованность изменений одновременно на уровне интерфейса и сервера, при этом доверенная проверка должна оставаться исключительно на backend.

12. Заключение

Настоящее руководство разработчика описывает внутреннюю организацию программного средства и принятые проектные решения. Система построена на современном технологическом стеке и имеет чёткое разделение ответственности между клиентской частью, серверной частью, реляционной БД и файловым хранилищем. Это создаёт основу для дальнейшего расширения функциональности и сопровождения программного изделия.